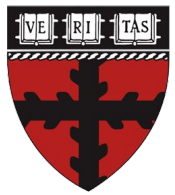
Lecture 1: Virtual Environments and Virtual Machines

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Outline

1. Motivation
2. Virtual Machines
3. Virtual Environments

Outline

1. **Motivation**
2. Virtual Machines
3. Virtual Environments

The AI Paradox: High Ambition vs. Execution Gaps

Summary of Enterprise Hurdles across Adoption, Culture, and Financial Impact

1. Adoption vs. Deployment

Despite massive growth, scaling and deployment remain a core technical challenge.

~25% **YoY Growth in AI Use**
McKinsey

Deployment Lag
Over 68% of companies take anywhere from 8 to 90+ days to deploy a single AI model.

2. Strategy & Culture Gap

Organizations heavily prioritize AI, but lack the structural roadmap to execute effectively.

25% **Enterprise Strategy**
IDC Research

Culture vs. Reality
While 2/3 aim for an "AI First" culture, only a quarter of active AI users have an enterprise-wide strategy.

3. Limited Business Impact

The commercial reality of AI pilots rarely scales to bottom-line performance.

>80% **No Tangible EBIT Effect**
GenAI Initiatives

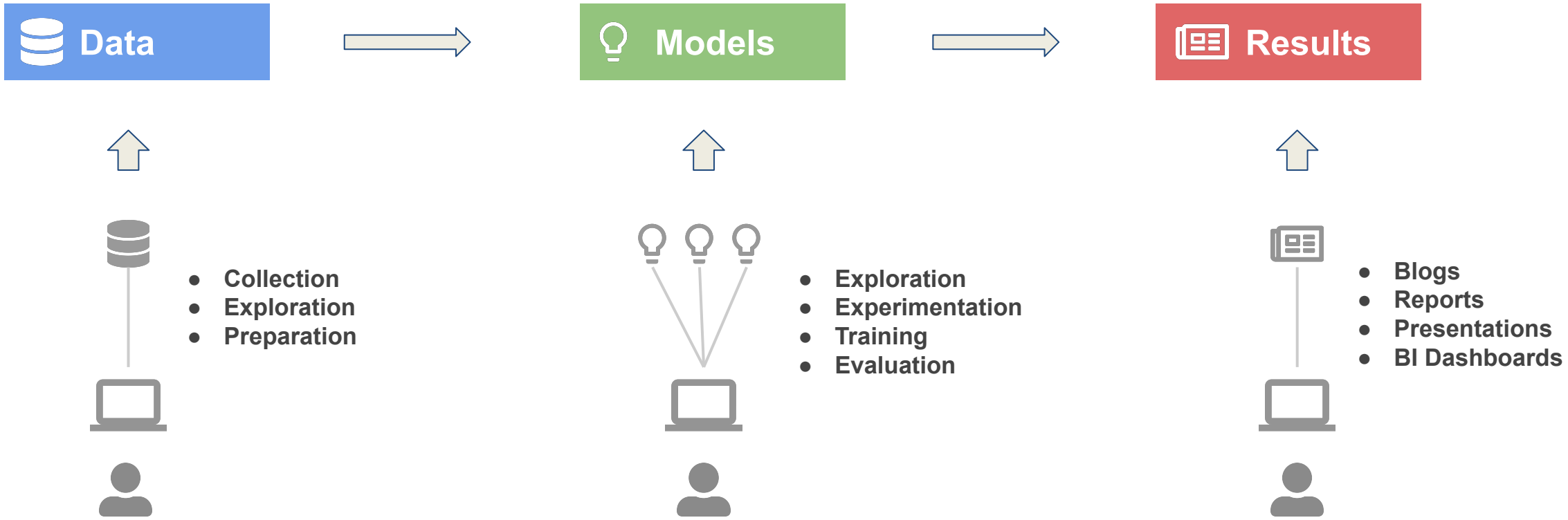
The Scaling Challenge
A vast majority of organizations successfully pilot models, but scaling them to drive true enterprise value fails.

Sources: McKinsey state of AI (2023) | IDC Enterprise AI Strategy | VentureBeat

Data Science Series to Real World

Data Science Series CS1090 A/B.

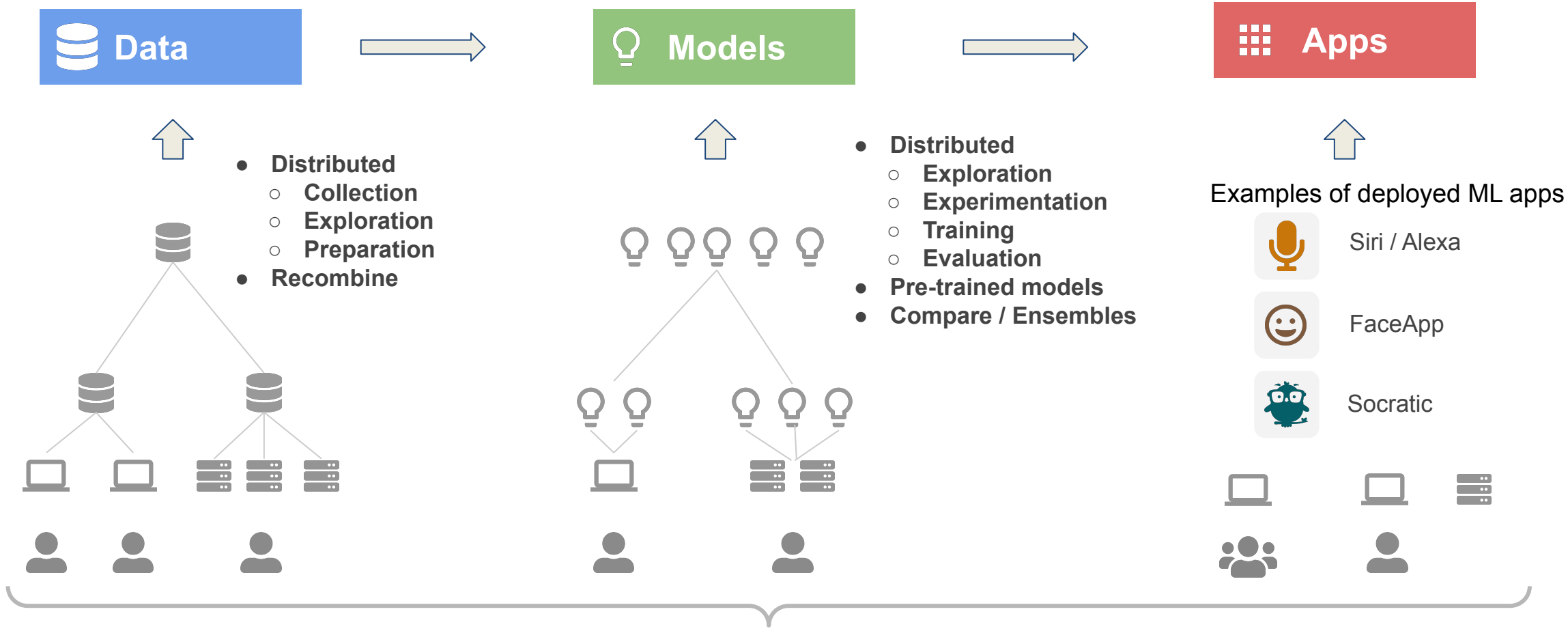
Prototype Workflow:



Single developer on one computer. Projects are individual to 2-3 member team.

Data Science Series to Real World

Real World



Team of developers on single/ multi node clusters on a Cloud Platform. Projects are 5+ member teams

Data Science Series to Real World (cont)

Key Challenges in Real-World Production Environments:

01. Team Integration

Onboarding new team members efficiently into existing data science workflows and codebases.

02. Environment Setup

Managing complex local installations, dependencies, and keeping development environments aligned.

03. Code Collaboration

Establishing robust version control, branches, and code review systems for machine learning models.

04. Sharing Assets

Securely distributing, storing, and tracking metadata for massive datasets and deep learning models.

05. Pipeline Automation

Automating data collection, training schedules, evaluation steps, and validation of updated models.

06. Production Discrepancies

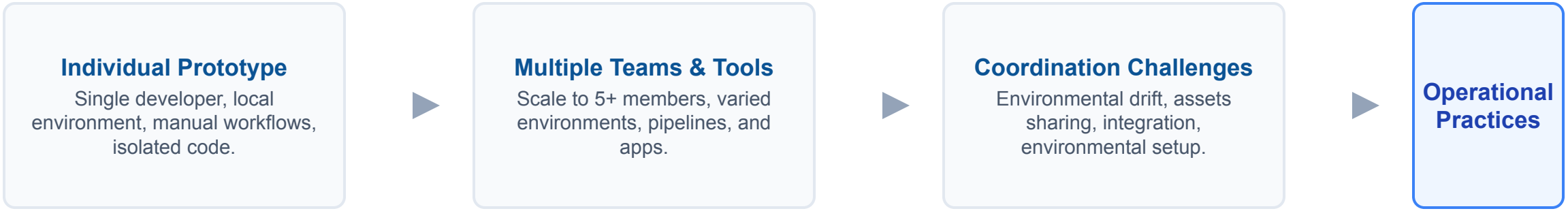
Resolving the infamous "it works on my machine" issues during local-to-production deployment phases.

Transitioning from Individual Playgrounds to Enterprise Infrastructure

Standardizing infrastructure through automation, containers, and **deployment operations (Ops)** becomes absolutely essential to scale team efforts and solve environmental drift.

From Production Challenges to MLOps

The six challenges are **not separate problems**. They are **coordination problems** created when machine-learning work moves from an individual prototype to a production system involving many people, environments, and tools.



MLOps provides the operating model for connecting data, models, software, and infrastructure so ML systems can be deployed and maintained reliably.

Ops for Machine / Deep Learning

Development Operations (DevOps):

Unifies software development (Dev) & operations (Ops) to deliver reliable software faster.

ML Operations (MLOps):

Integrates ML/DL model development with app development & operations.

MLOps - Tasks

Machine / Deep Learning

- Data collection & exploration
- Model exploration & selection
- Training & evaluation
- Distillation & compression

Application Dev

- APIs / Model serving
- ML integration
- Web & mobile apps
- Edge device apps
- Automation scripts

Operations

- Provisioning & managing servers
- Maintain 100% uptime of app/apis
- CI/CD Integration
- Continuous Data Collection
- Model/data monitoring & versioning
- ML Workflow Management

MLOps - Tech Stack

Data	Models	Development	Operations

MLOps - Tech Stack

Data

- Spark
- Hadoop
- Kafka
- Bigquery
- Airflow
- Vertex AI Pipelines
- DVC
- Postgres
- Neo4j
- Chroma

Models

Development

Operations

MLOps - Tech Stack

Data

- Spark
- Hadoop
- Kafka
- Bigquery
- Airflow
- Vertex AI Pipelines
- DVC
- Postgres
- Neo4j
- Chroma

Models

- PyTorch
- TensorFlow
- Hugging Face
- Scikit-learn
- MLflow
- Weights & Biases
- XGBoost
- ONNX

Development

Operations

MLOps - Tech Stack

Data

- Spark
- Hadoop
- Kafka
- Bigquery
- Airflow
- Vertex AI Pipelines
- DVC
- Postgres
- Neo4j
- Chroma

Models

- PyTorch
- TensorFlow
- Hugging Face
- Scikit-learn
- MLflow
- Weights & Biases
- XGBoost
- ONNX

Development

- VS Code
- Jupyter
- Git
- GitHub
- Docker
- Conda
- pip / poetry

Operations

MLOps - Tech Stack

Data

- Spark
- Hadoop
- Kafka
- Bigquery
- Airflow
- Vertex AI Pipelines
- DVC
- Postgres
- Neo4j
- Chroma

Models

- PyTorch
- TensorFlow
- Hugging Face
- Scikit-learn
- MLflow
- Weights & Biases
- XGBoost
- ONNX

Development

- VS Code
- Jupyter
- Git
- GitHub
- Docker
- Conda
- pip / poetry

Operations

- AWS
- GCP
- Kubernetes
- Ansible
- Jenkins
- GitHub Actions
- Cloud Functions
- Pulumi

MLOps - Tech Stack

Data	Models	Development	Operations
Spark	PyTorch	VS Code	AWS
Hadoop	TensorFlow	Jupyter	GCP
Kafka	Hugging Face	Git	Kubernetes
Bigquery	Scikit-learn	GitHub	Ansible
Airflow	MLflow	Docker	Jenkins
Vertex AI Pipelines	Weights & Biases	Conda	GitHub Actions
DVC	XGBoost	pip / poetry	Cloud Functions
Postgres	ONNX		Pulumi
Neo4j			
Chroma			



Data Engineers	Data Scientists	Software Engineers	Systems Engineers

Our Stack

AI / ML

Data:
Chroma, PostgreSQL
DVC, Label Studio

Framework:
TensorFlow
PyTorch

ML Tasks:
Google Colab
Vertex AI

Tracking:
W&B
Custom

Development

APIs & Tools:
FastAPI, FastMCP
Model Serving

LLM APIs:
OpenAI
Gemini

Frontend:
React, NextJS

IDE:
VS Code, Cursor
IDE of choice

Operations

Source Control:
GitHub

Containerization:
Docker

Cloud Provider:
Google Cloud Platform

Continuous Integration/ Deployment:
GitHub Actions, Pulumi

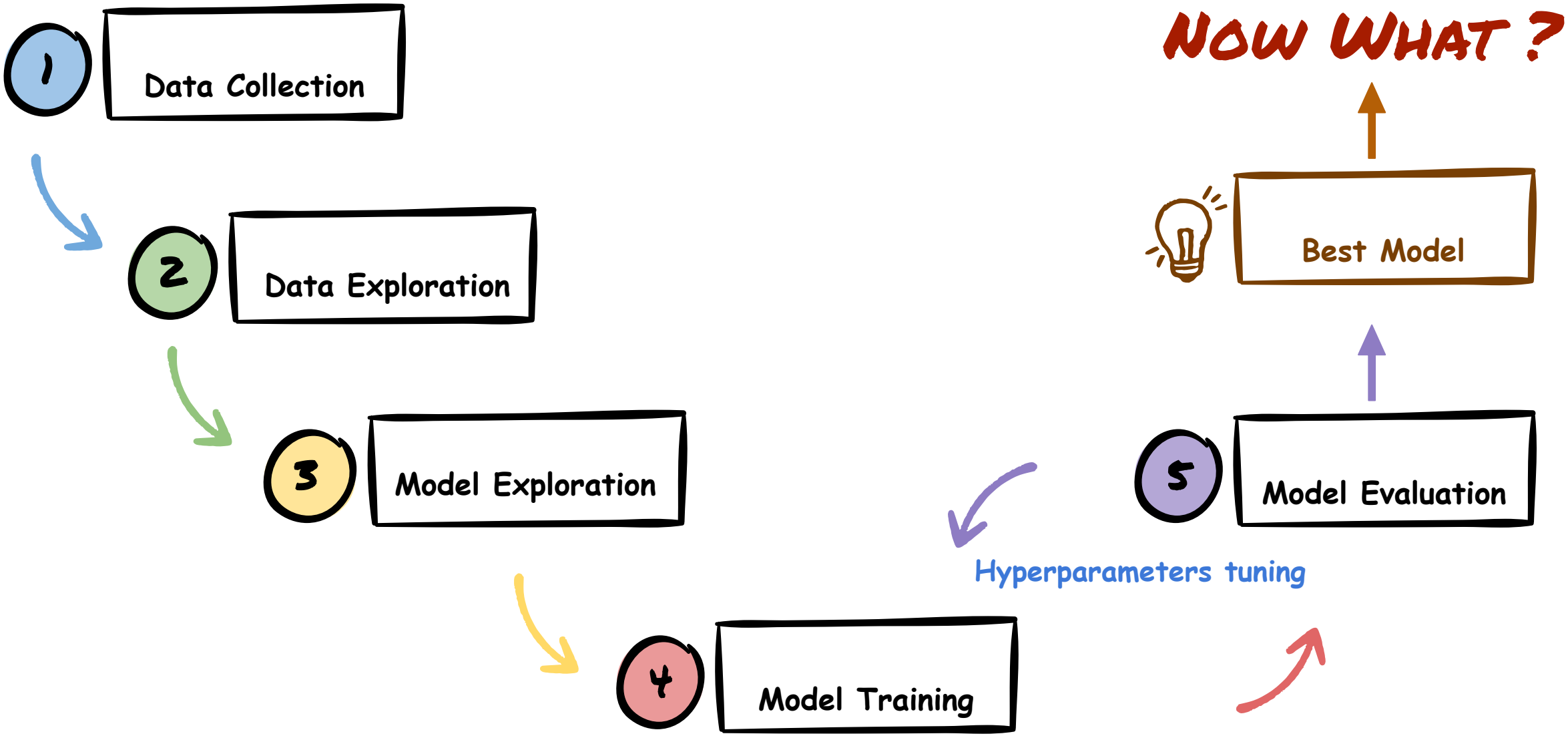
Scaling:
Kubernetes

TRANSITION

Motivation: Deep Learning Flow

Moving from model evaluation to the next phase of the lifecycle

Motivation: Deep Learning Flow



Learning by Doing: Building a Real Application

Guided Example Project

- Throughout the semester, the teaching team will develop Formaggio, our reference project.
- Each week, we will connect one part of the system to the course topic.
- You will apply the same ideas to your own project.



Parmigiano



Ricotta



Feta



Brie

What is Formaggio.me?

- Pavlos loves cheese and always serves it when he hosts a party.
- He wants to build an app that identifies cheese types and explains how they are made, where they come from, and what they pair well with.
- Project Summary



Parmigiano



Ricotta



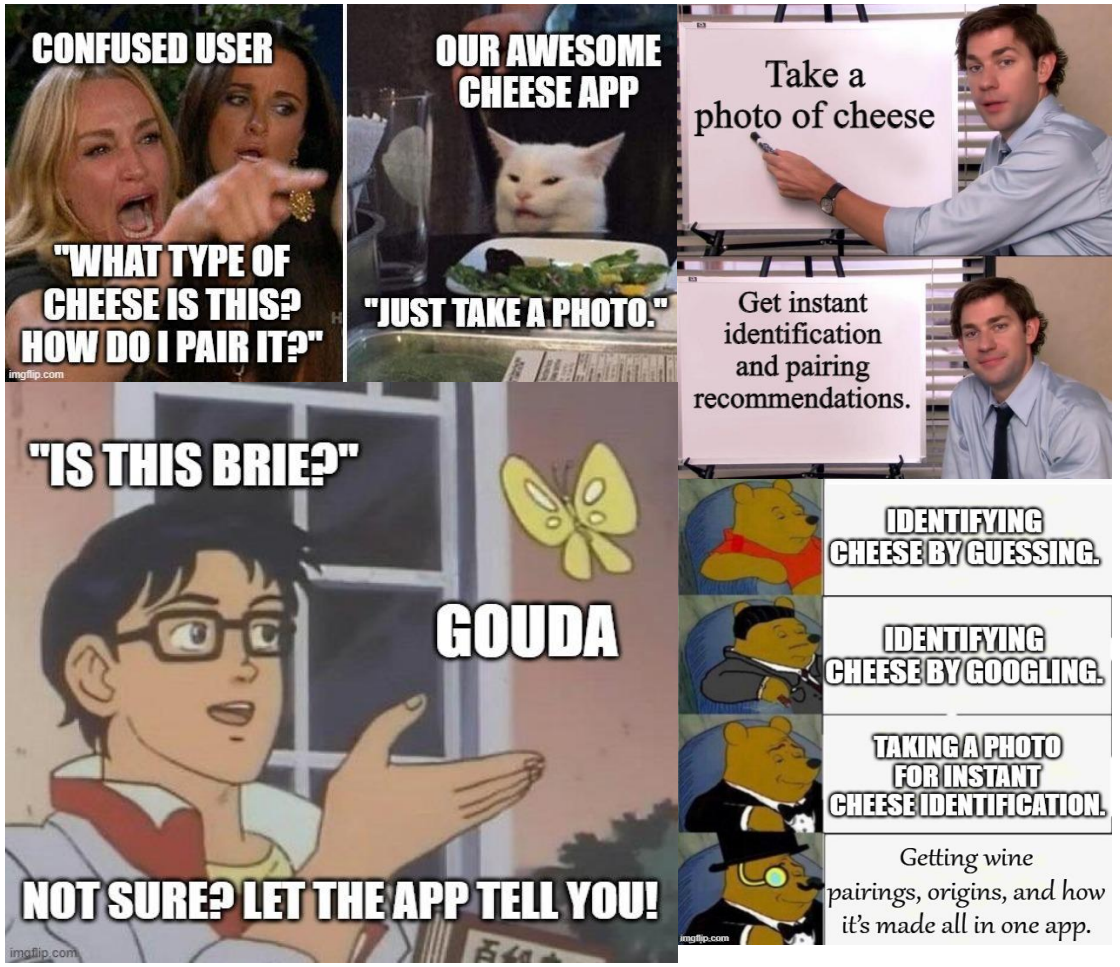
Feta



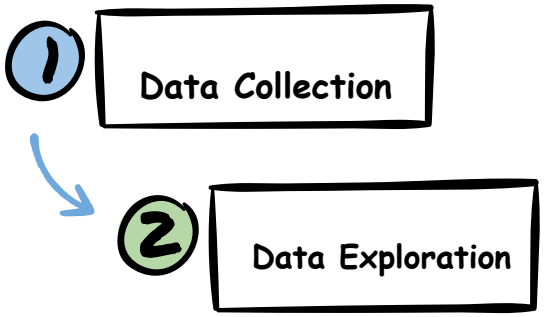
Brie

Functional Requirements

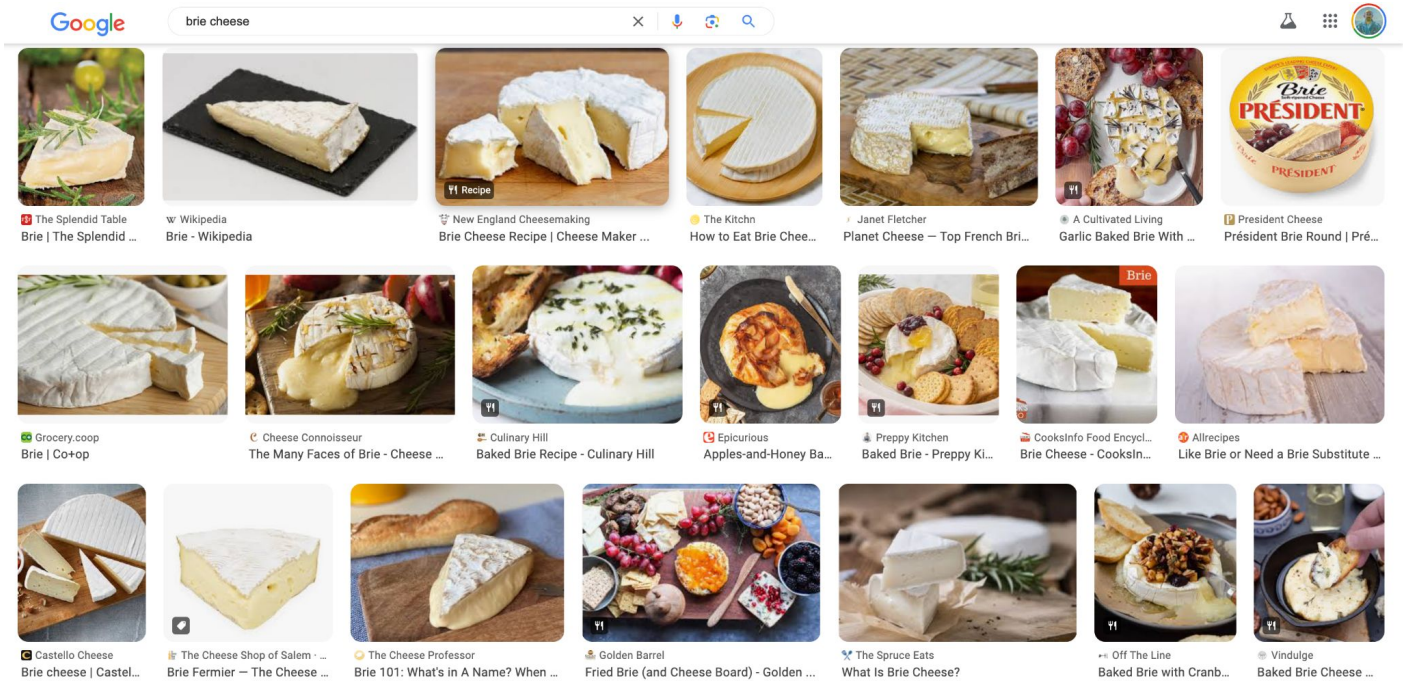
- **Take a photo** of a cheese and identify it.
- **Ask** about its origin, how it is made, wine pairings, and where to buy it.



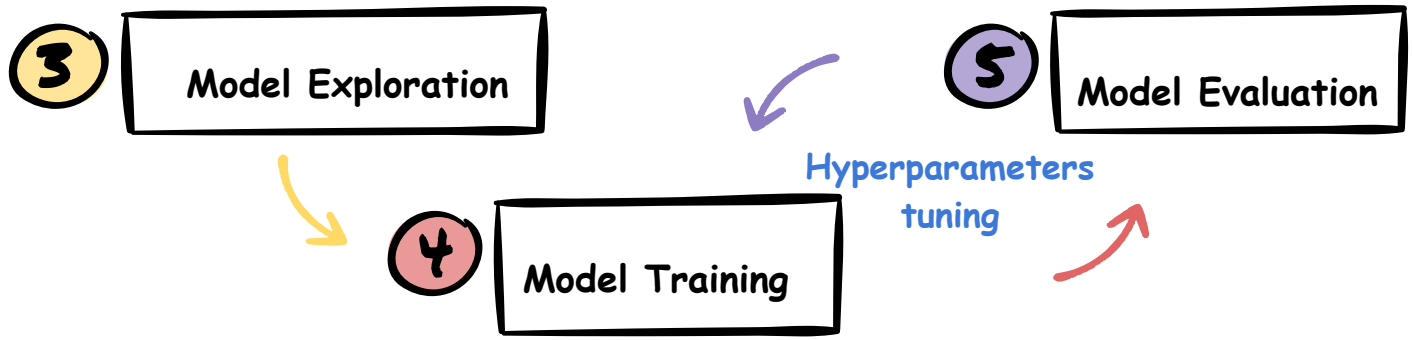
🧀 Cheese App: Data



- Collect images from Google/Bing
- For our app we downloaded images for cheeses brie, parmigiano, gouda, gruyere
- Images are organized into 4 labels



🧀 Cheese App: Models



- Identify our problem task
- Try various model architectures
- Transfer learning
- Hyperparameter tuning
- Experiment tracking

trainable_parameters	execution_time	loss	accuracy
2,306,051	2.97 mins	42.87	90.91%
82,179	3.19 mins	42.79	90.30%
164,355	3.91 mins	70.97	89.09%
2,388,227	2.95 mins	82.03	88.48%
11,112,323	6.85 mins	0.79	67.88%
25,950,531	8.19 mins	0.74	66.67%
22,514,755	4.78 mins	1.07	41.21%

🧀 Cheese App: Choose the Best Model

Best Model

trainable_parameters	execution_time	loss	accuracy	model_size	learning_rate	batch_size	epochs	optimizer	name
2,306,051	2.97 mins	42.87	90.91%	10 MB	0.001	32	10	SGD	tfhub_mobilenetv2_train_base_True
82,179	3.19 mins	42.79	90.30%	10 MB	0.001	32	10	SGD	tfhub_mobilenetv2_train_base_False
164,355	3.91 mins	70.97	89.09%	10 MB	0.001	32	15	SGD	mobilenetv2_train_base_False
2,388,227	2.95 mins	82.03	88.48%	10 MB	0.001	32	10	SGD	mobilenetv2_train_base_True
11,112,323	6.85 mins	0.79	67.88%	44 MB	0.010	32	25	SGD	4_block
25,950,531	8.19 mins	0.74	66.67%	104 MB	0.010	32	25	SGD	2_block
22,514,755	4.78 mins	1.07	41.21%	90 MB	0.010	32	15	SGD	vgg_style

Selection is based on multiple criteria, including accuracy, memory usage, execution time, and other key performance metrics

NOW WHAT ?

Steps to build an App?



- Expose best model as an API
- Build a frontend
- Integrate model prediction API into the app
- Deploy app to a cloud provider
- Go live: http://formaggio.me

How do we build an App?

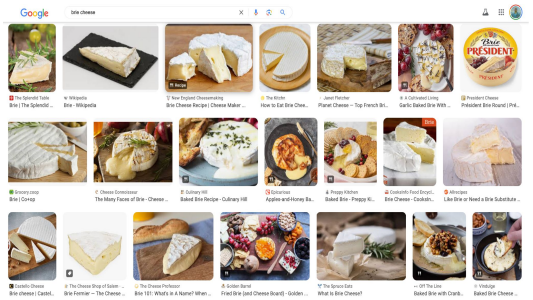
Data Collection

Data Exploration

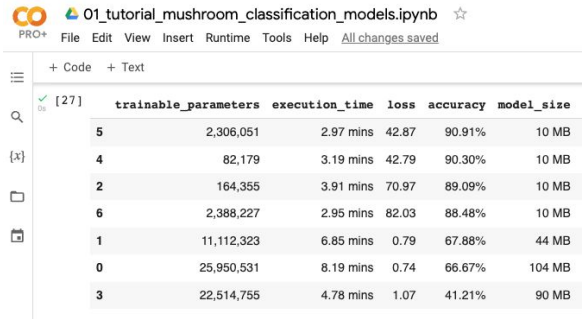
Model Exploration

Model Training

Model Evaluation



Python Script

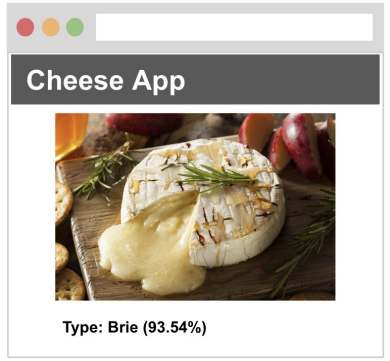


	trainable_parameters	execution_time	loss	accuracy	model_size
5	2,306,051	2.97 mins	42.87	90.91%	10 MB
4	82,179	3.19 mins	42.79	90.30%	10 MB
2	164,355	3.91 mins	70.97	89.09%	10 MB
6	2,388,227	2.95 mins	82.03	88.48%	10 MB
1	11,112,323	6.85 mins	0.79	67.88%	44 MB
0	25,950,531	8.19 mins	0.74	66.67%	104 MB
3	22,514,755	4.78 mins	1.07	41.21%	90 MB

Colab

Rest API

Best Model

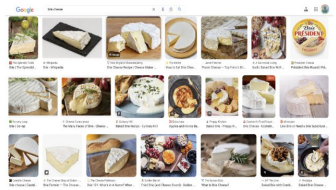




IDE / Code Editor

Challenges

Development



Python Script



Colab

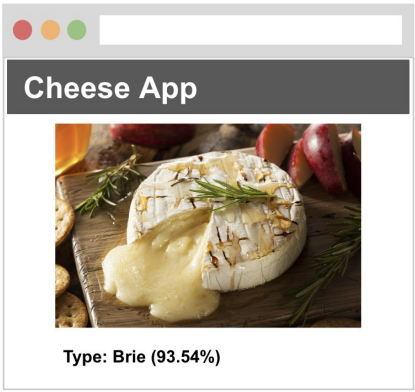
Rest API

Best Model



IDE / Code Editor

Deployment



Python: uv
Chromium: Mac install
OS: Mac

Python: Colab provided env
OS: Linux

Python: uv
OS: Mac

Python: uv
OS: Linux

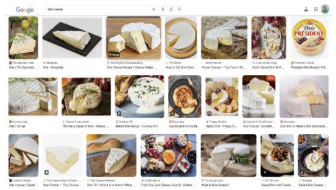


One developer
Using a Macbook

Server

Challenges - Multiple Developers

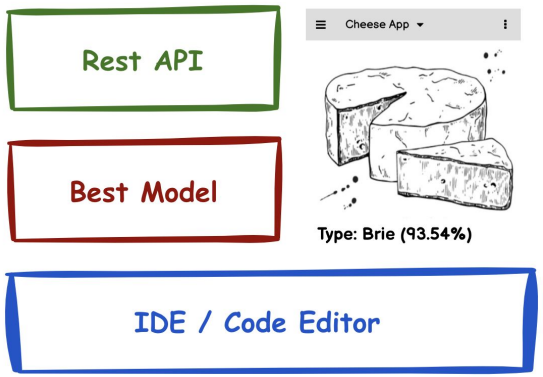
Development



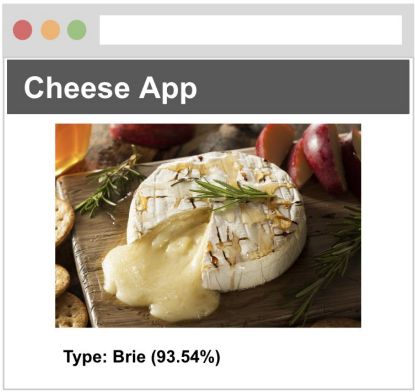
Python Script



Colab



Deployment



Python: uv
Chromium: Mac install
OS: Mac

Python: Colab provided env
OS: Linux

Python: uv
OS: Mac

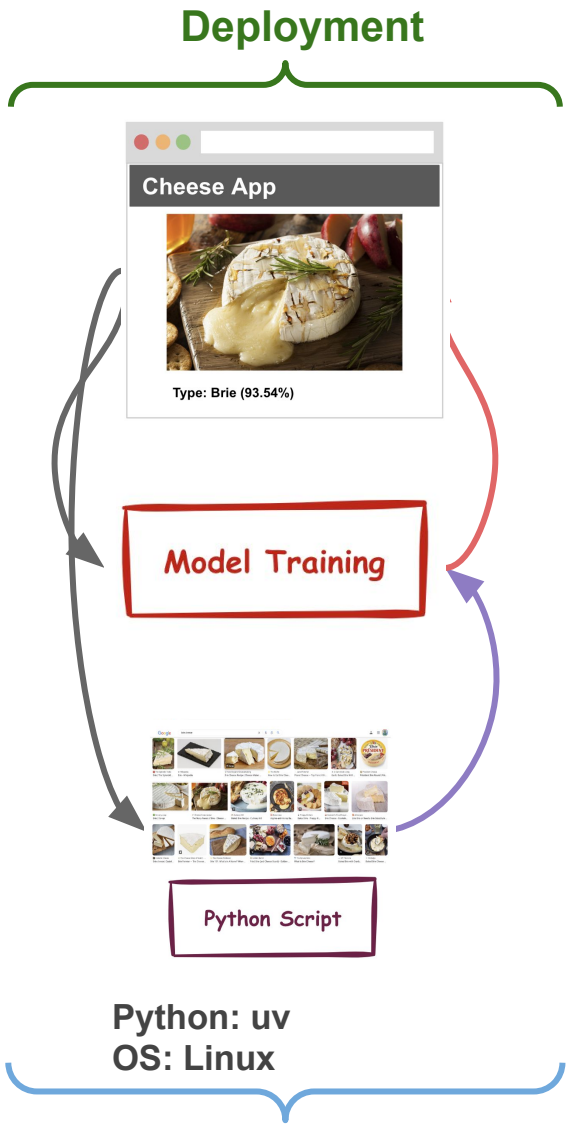
Python: uv
OS: Linux

Server



Multiple developers, Using Mac and Windows OS

Challenges - Multiple Developers + Automation



Key Challenges in Real-World Production Environments:

Formaggio now faces the same production challenges we identified earlier

01. Team Integration

Onboarding new team members efficiently into existing data science workflows and codebases.

02. Environment Setup

Managing complex local installations, dependencies, and keeping development environments aligned.

03. Code Collaboration

Establishing robust version control, branches, and code review systems for machine learning models.

04. Sharing Assets

Securely distributing, storing, and tracking metadata for massive datasets and deep learning models.

05. Pipeline Automation

Automating data collection, training schedules, evaluation steps, and validation of updated models.

06. Production Discrepancies

Resolving the infamous "it works on my machine" issues during local-to-production deployment phases.

Challenges / **Solutions**

Solutions:

- Isolate development into environments that can be shared
- Develop in a common OS regardless of developers host OS
- Track software/framework installs

Tools



Virtual Machines



Virtual Environments



Containers

We will cover these next week

Tools



Virtual Machines



Virtual Environments

We will cover these later



Containers

We will cover these next week

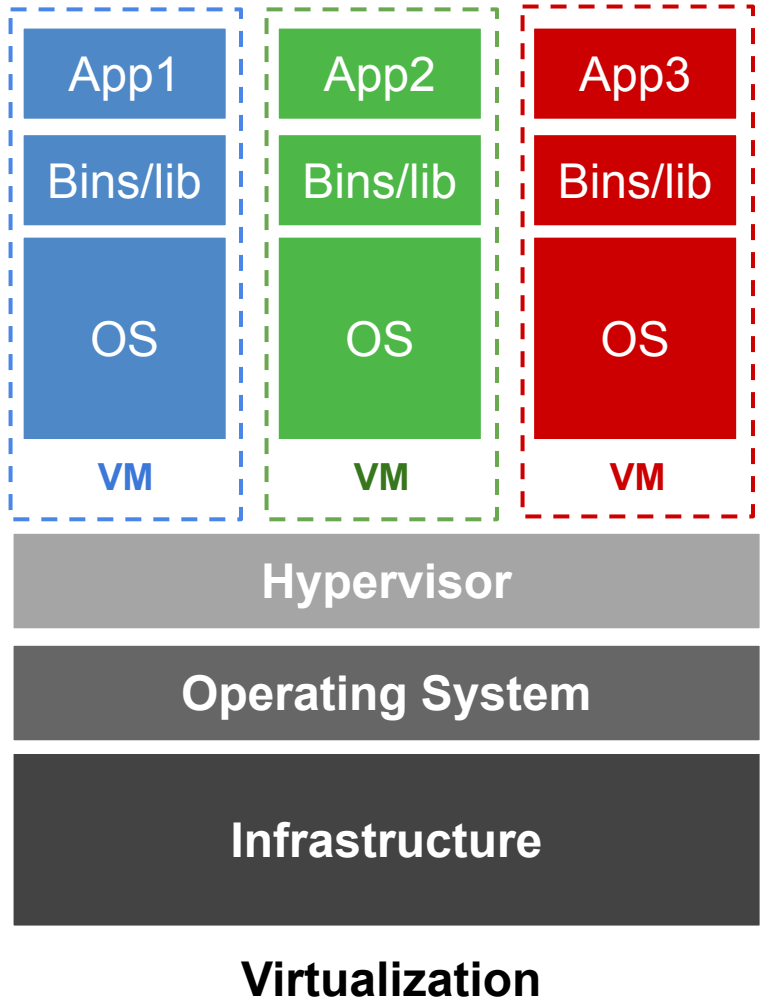
Motivation

- **Uniform Operating Environments**: The desire for a standardized OS across all team member workstations.
- **Consistent Software Configuration:** The need for identical software setups across the team.
- **Effortless Instance Management:** The need for simple procedures to instantiate and terminate virtual machines.

Virtual Machines!

What is a Virtual Machine?

- Virtual machines mimic real hardware like CPUs and hard drives.
- Hypervisor is a software that manages (creates and runs) virtual machines on a physical machine.
- Unlimited VMs can be run, subject to hardware limits.
- The main OS on the physical machine is called the "host," and VMs run "guest" OS.
- Guest VMs can have different operating systems.



Virtualization

Why should we use virtual machines?

Advantages

- **Complete Autonomy:** it works like a separate computer system; it is like running a computer within a computer.
- **Enhance Security**: the software inside the virtual machine cannot affect the actual computer.
- **Cost-Effectiveness:** Purchase a single machine and run multiple operating systems.
- **Widely Adopted:** Utilized by all major cloud providers for on-demand server instances.

Software for Virtualization

- VirtualBox
- VMWare
- Parallels

Why shouldn’t we use virtual machines?

Limitations

- **Local Hardware Dependency:** Relies on the hardware resources of the host machine.
- **Limited Portability:** Large file sizes can impede easy transfer or deployment.
- **Resource Overhead:** Additional computational and memory resources are required to operate.
- **Reduced Performance:** The guest system typically runs slower than the host environment.
- **Slow Initialization:** Extended startup times compared to native systems.
- **Graphics Constraints:** May lack the graphical capabilities of the host system.

Virtual Machines Tutorial

France, Italy, Switzerland, Spain, and the Netherlands are known for distinctive cheese traditions. To serve a **global** audience, our app should support multiple languages.

We will begin with a simple translation tool built in Python with Google Translate. This first example provides a foundation for later work on implementation, deployment, and production AI systems.

Virtual Machines Tutorial

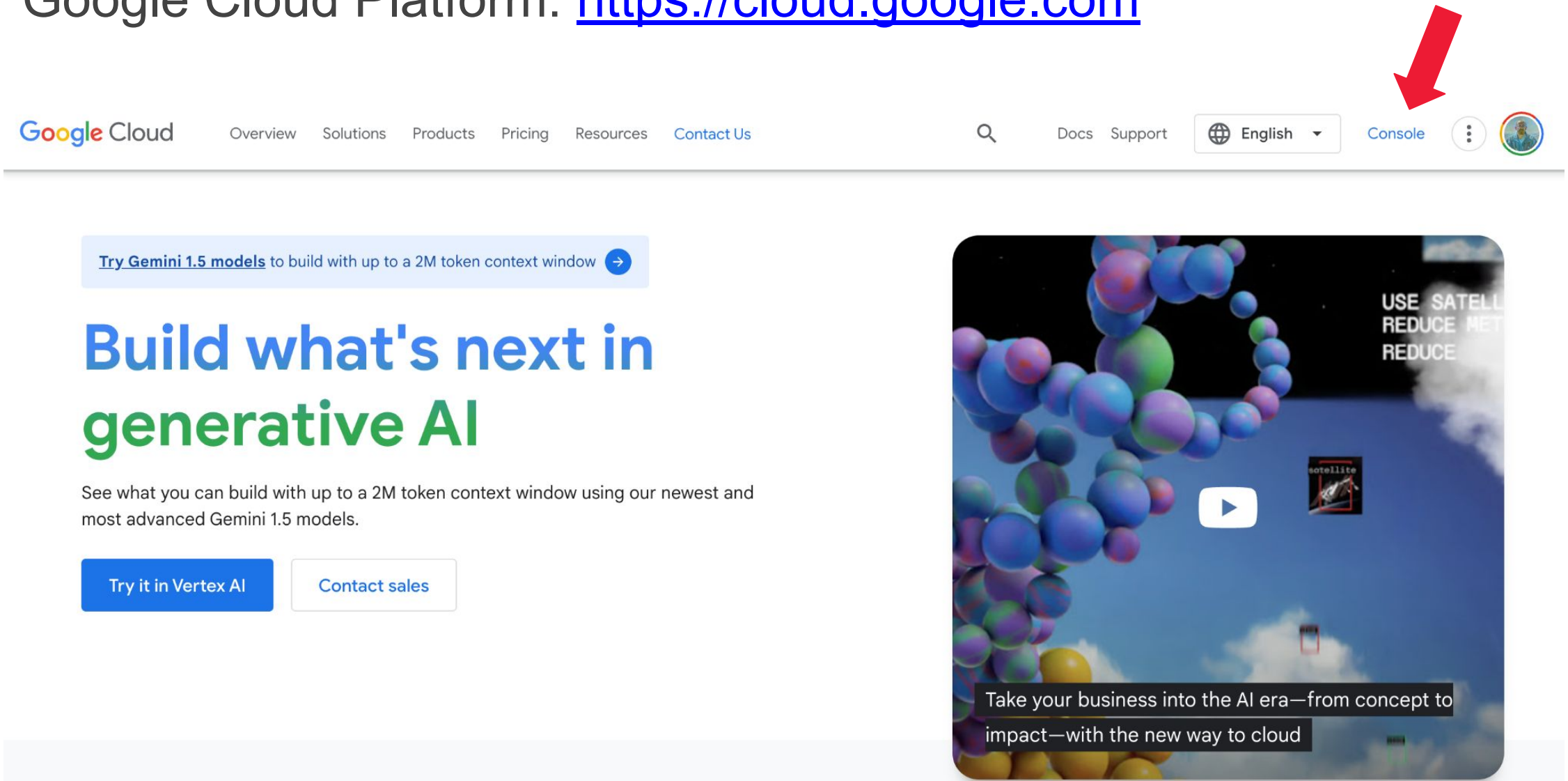
Running the Simple-Translate App on a **Virtual Machine**
To achieve this, follow the steps below:



- **Create** a Virtual Machine Instance.
- **SSH** into the Virtual Machine.
- **Install** Required Dependencies: git, Python.
- **Download** and Execute the Simple-Translate Python Script.
- For detailed instructions, please refer to the following link:
  Installing App on VM Manually.
  (https://github.com/dlops-io/simple-translate#installing-app-on-vm-manually)

Virtual Machines Tutorial

Google Cloud Platform: https://cloud.google.com



Creating a New Account



Email Control

You can use any email address you control to register your new Google Cloud account.

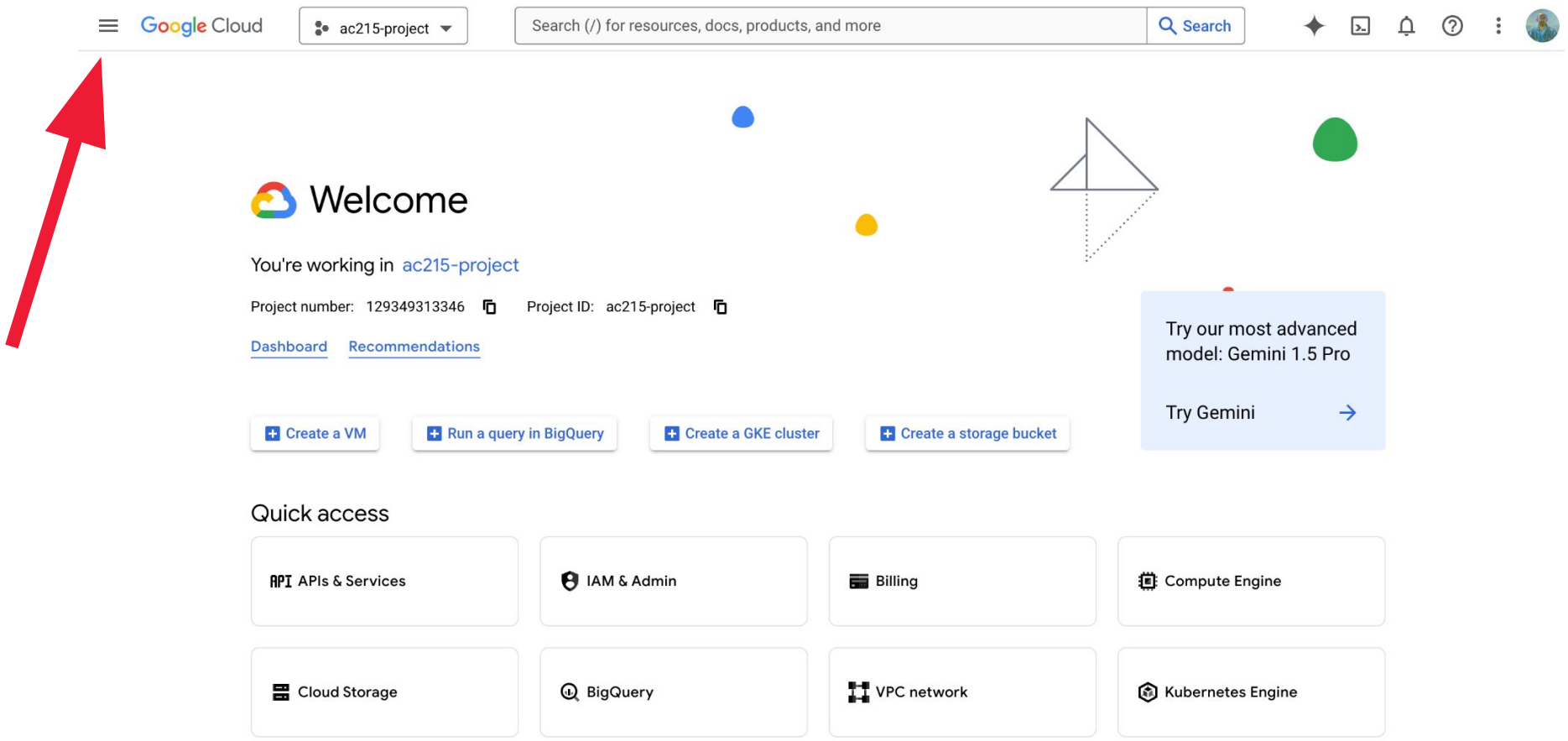


Personal Scope

Keep the account personal. Do not link or tie it to any uniaversity or employer domain.

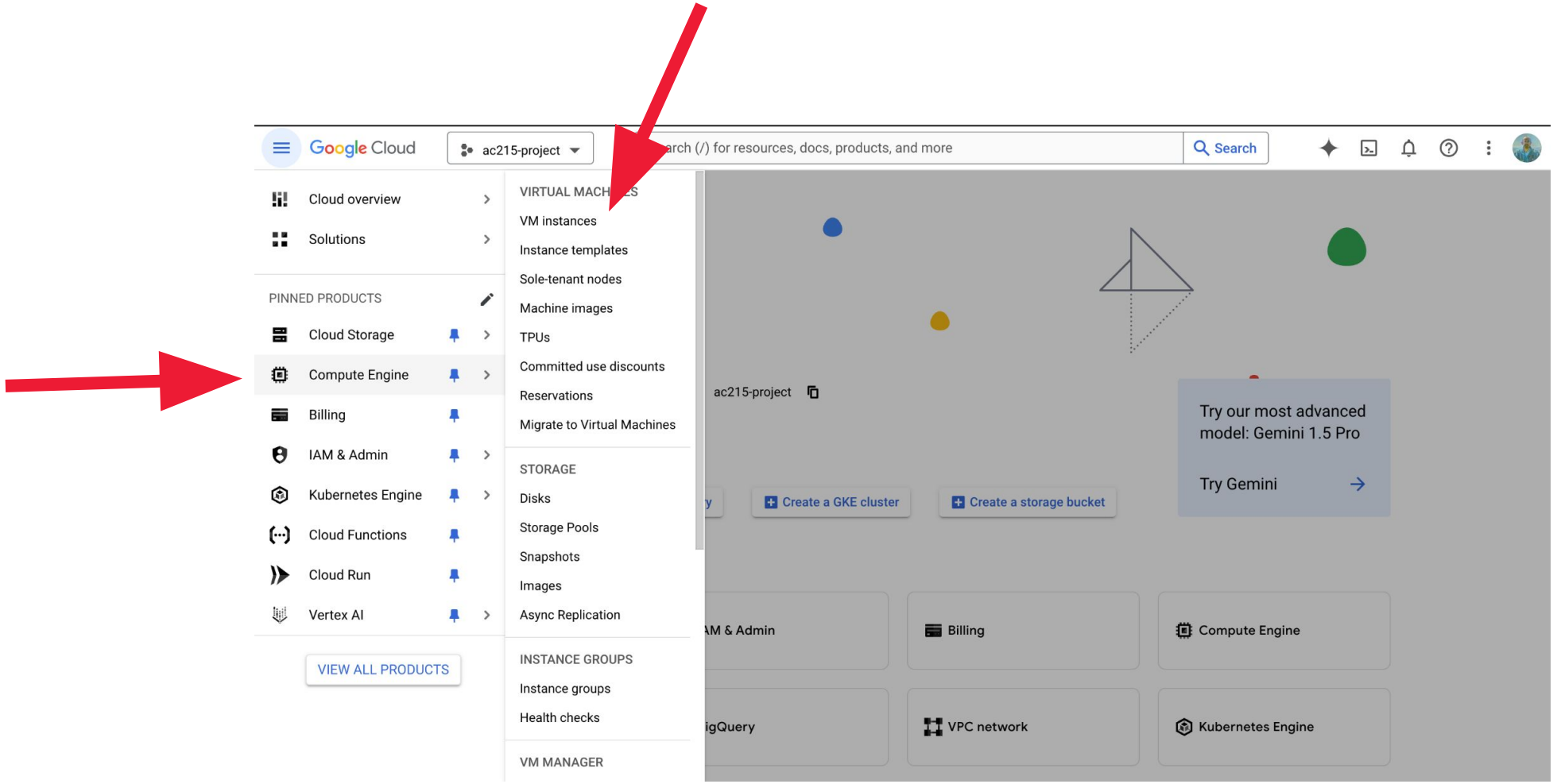
Virtual Machines Tutorial

Go to Navigation Menu



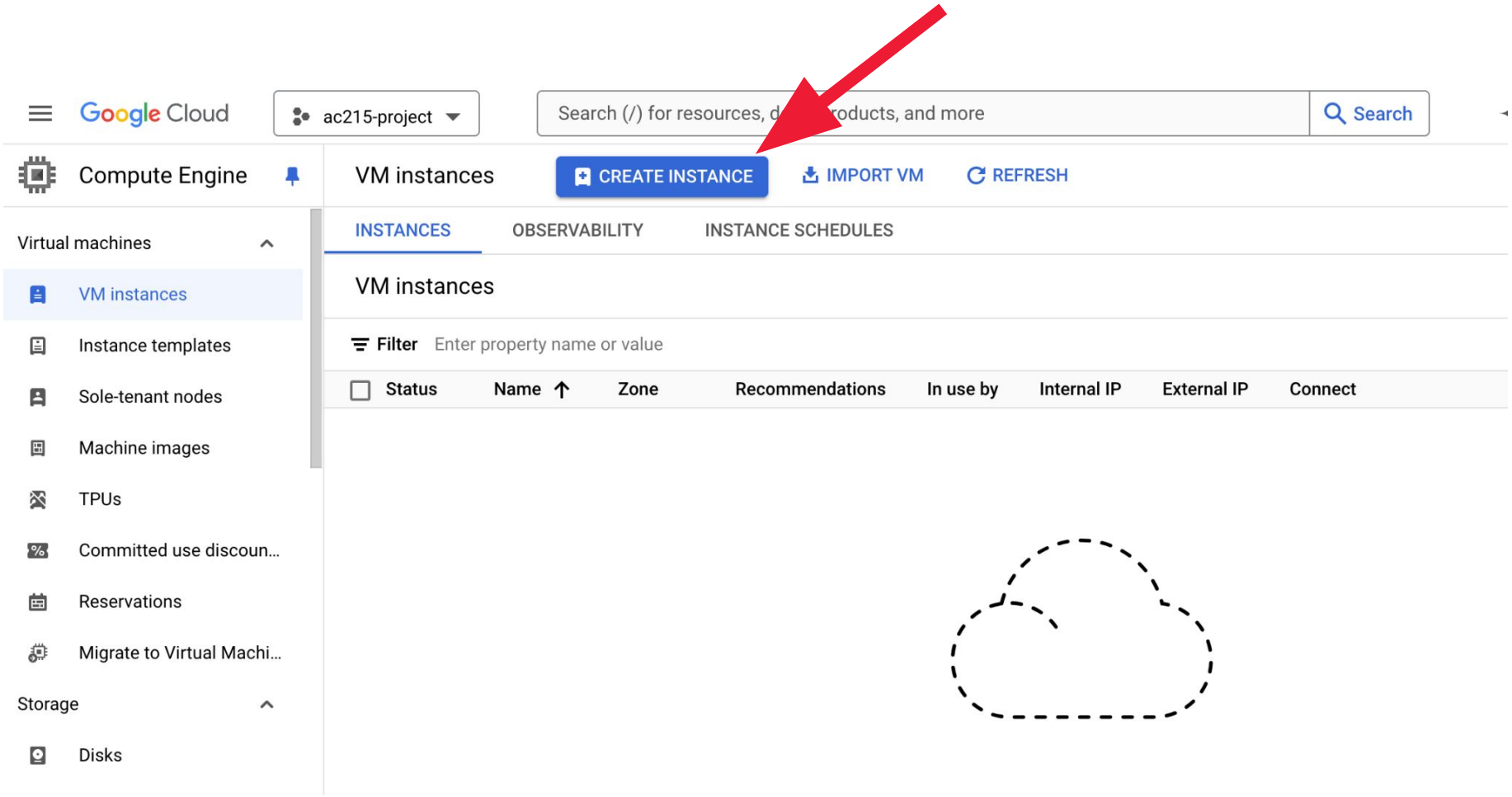
Virtual Machines Tutorial

Select compute engine and VM instances



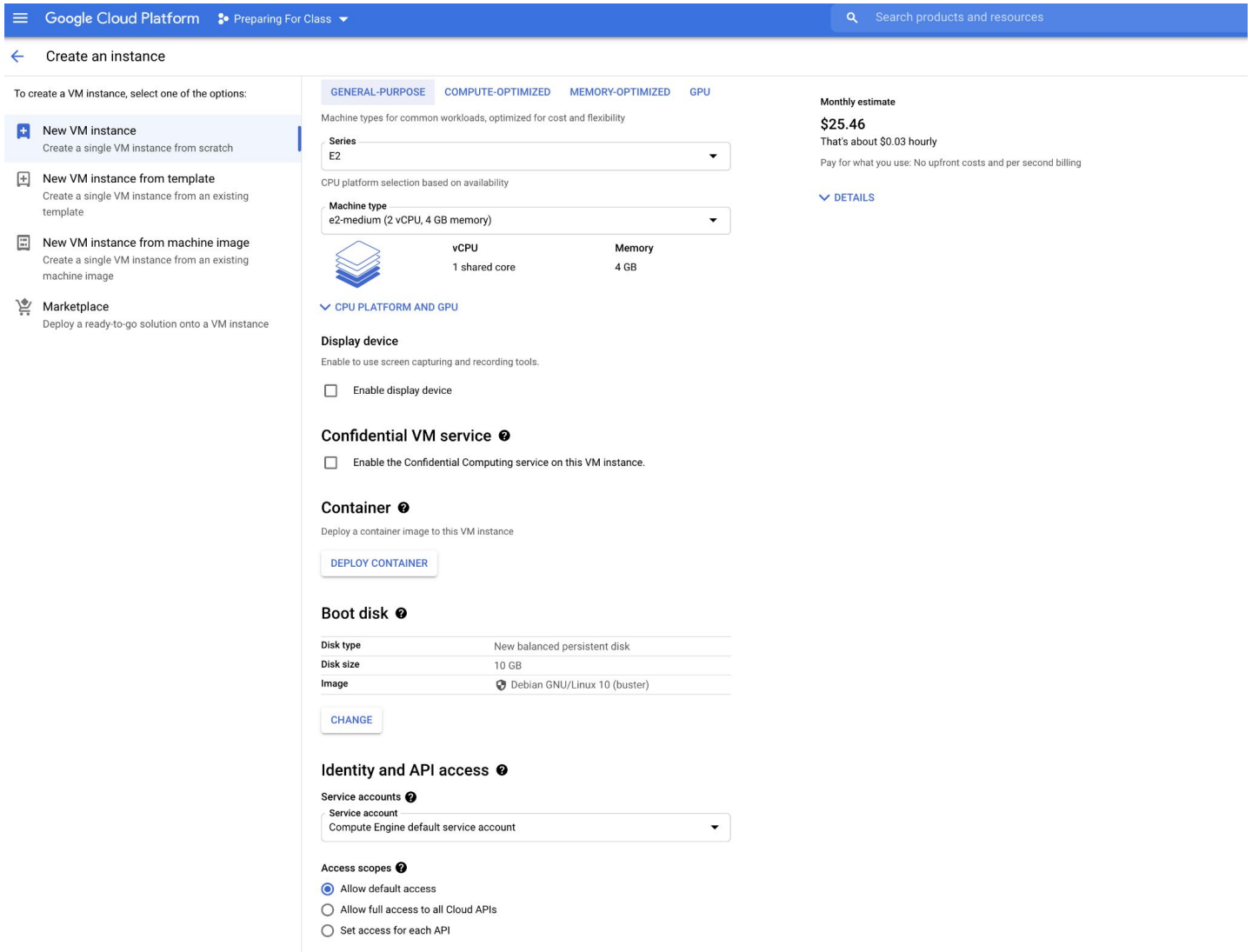
Virtual Machines Tutorial

You should see this (or similar). Click on CREATE INSTANCE



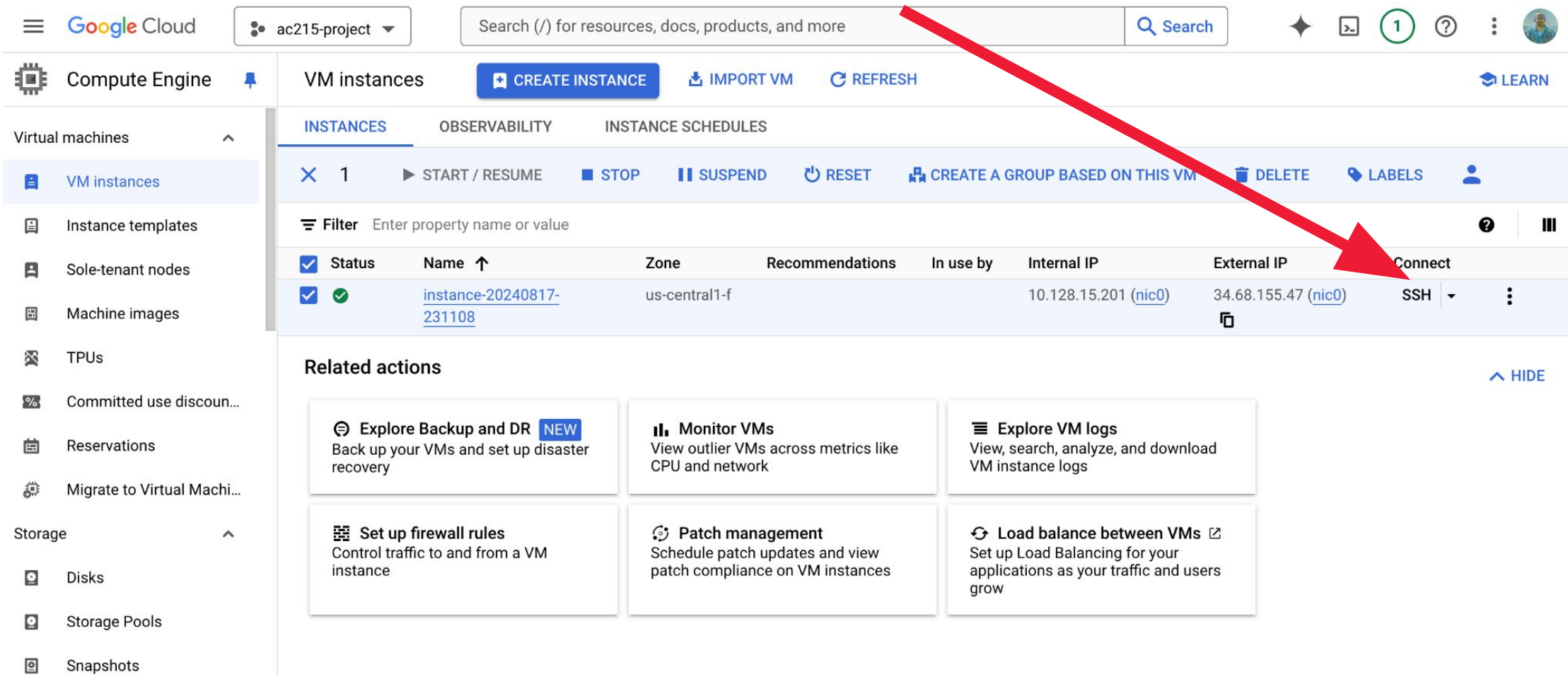
Virtual Machines Tutorial

Select all defaults



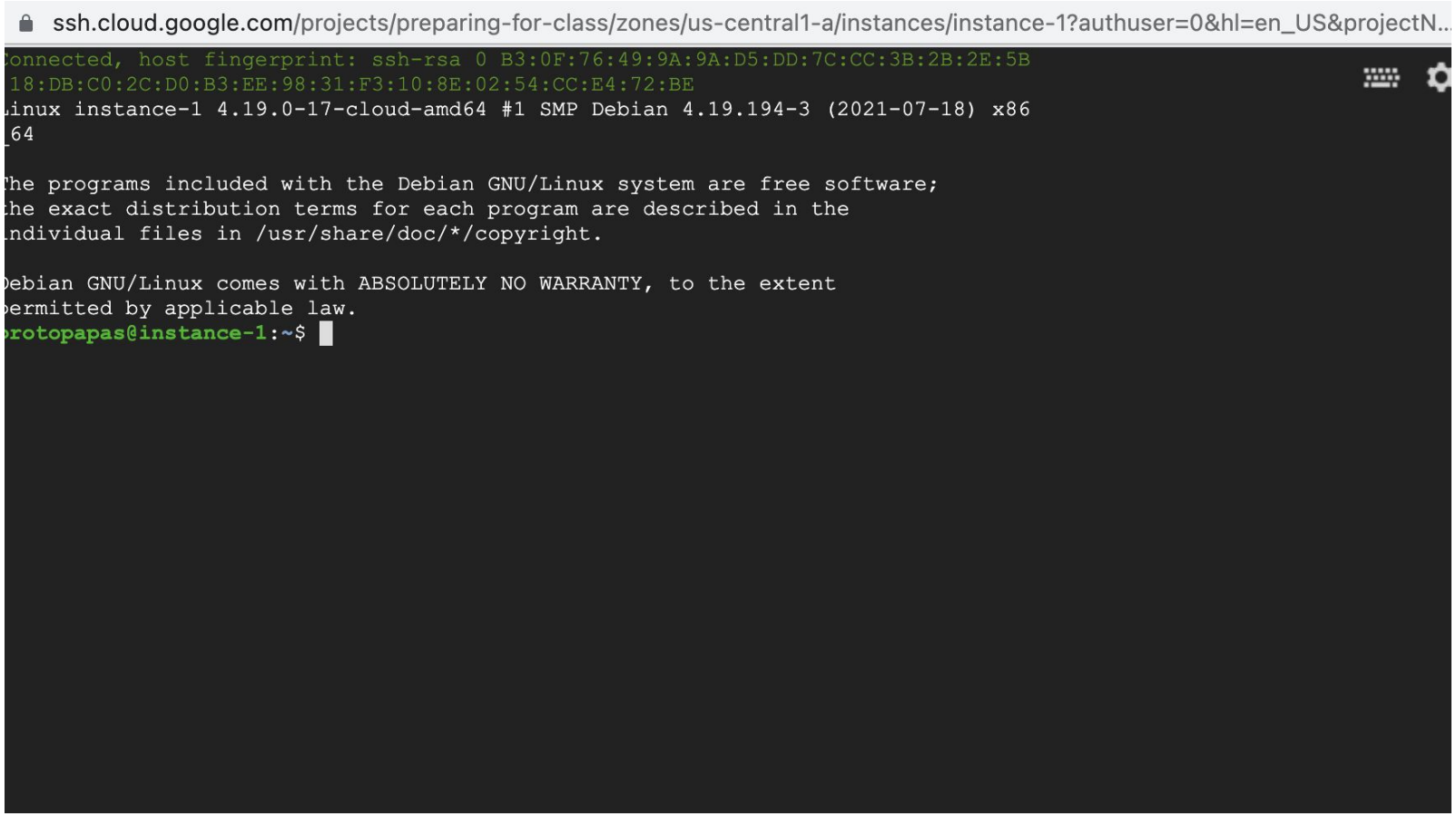
Virtual Machines Tutorial

Wait for instance to start and click on ssh



Virtual Machines Tutorial

And here is your virtual machine



git clone https://github.com/dlops-io/simple-translate.git

Tools



Virtual Machines



Virtual Environments



Containers

We will cover these next week

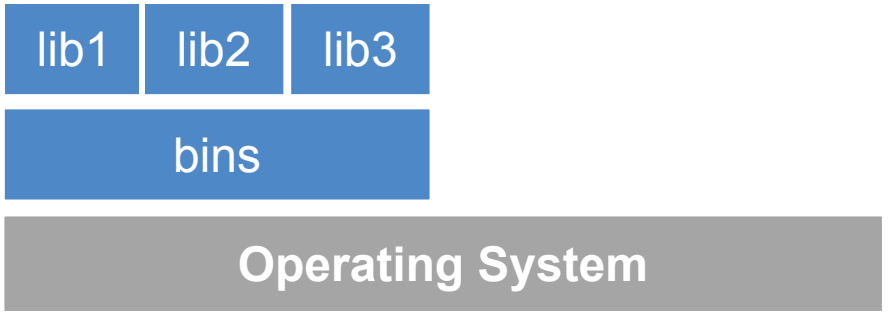
What are virtual environments

A virtual environment is an isolated Python setting in which the interpreter can execute libraries and scripts independently of other virtual environments.

- Consider a virtual environment as a directory containing the following components:
  - *`site_packages/`: A directory where third-party libraries are installed.*
  - *Symlinks: Links to system executables.*
  - *Scripts: These ensure that the code utilizes the interpreter and site packages specific to the virtual environment.*

Why should we use virtual environment?

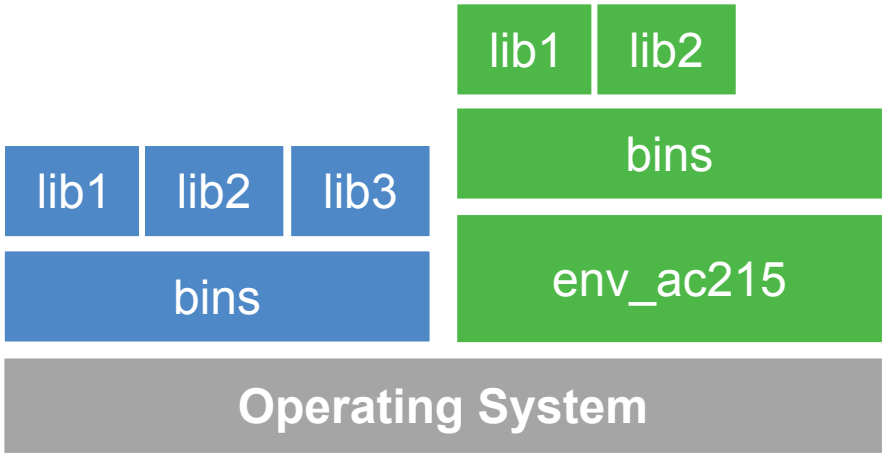
Maggie took CS1090B and used to run their Jupyter notebooks from the Anaconda prompt. Whenever they installed a module, it was placed in one of the following folders: `bin`, `lib`, `share`, or `include`. They could then import the module and used it without any issue.




```
$ which python
/c/Users/maggie/Anaconda3/python
```

Why should we use virtual environment?

Maggie begins taking AC215 and decides that isolating the new coding environment from previous ones would be beneficial to avoid package conflicts. To achieve this, they employ a layer of abstraction known as a virtual environment. This helps them keep modules organized and prevents issues while developing new projects.

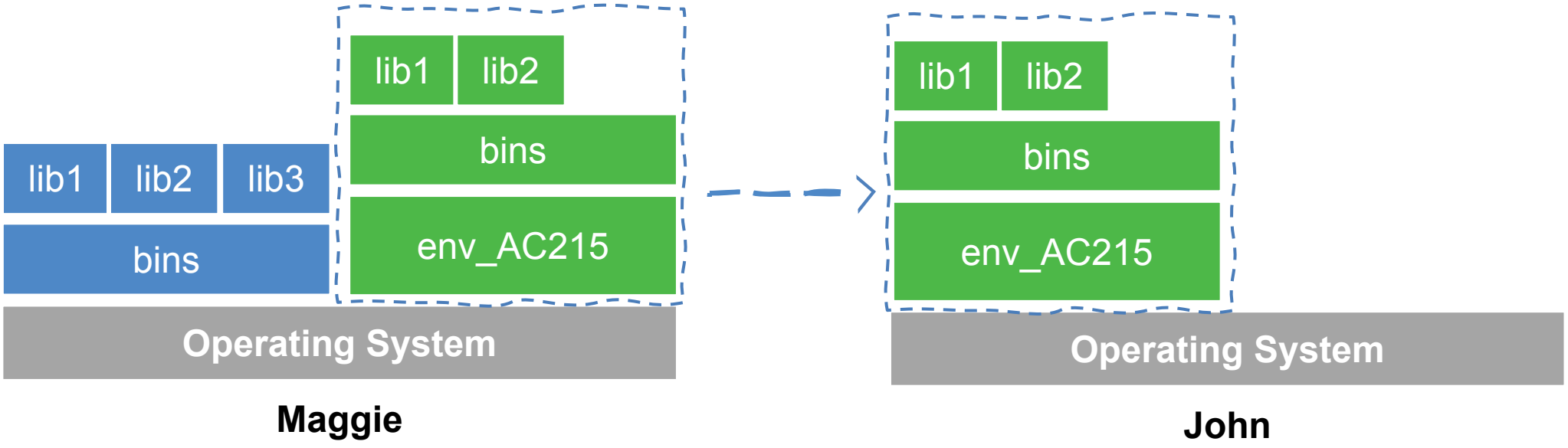


Maggie

$ which python
/c/Users/maggie/Anaconda3/envs/env_ac215/python

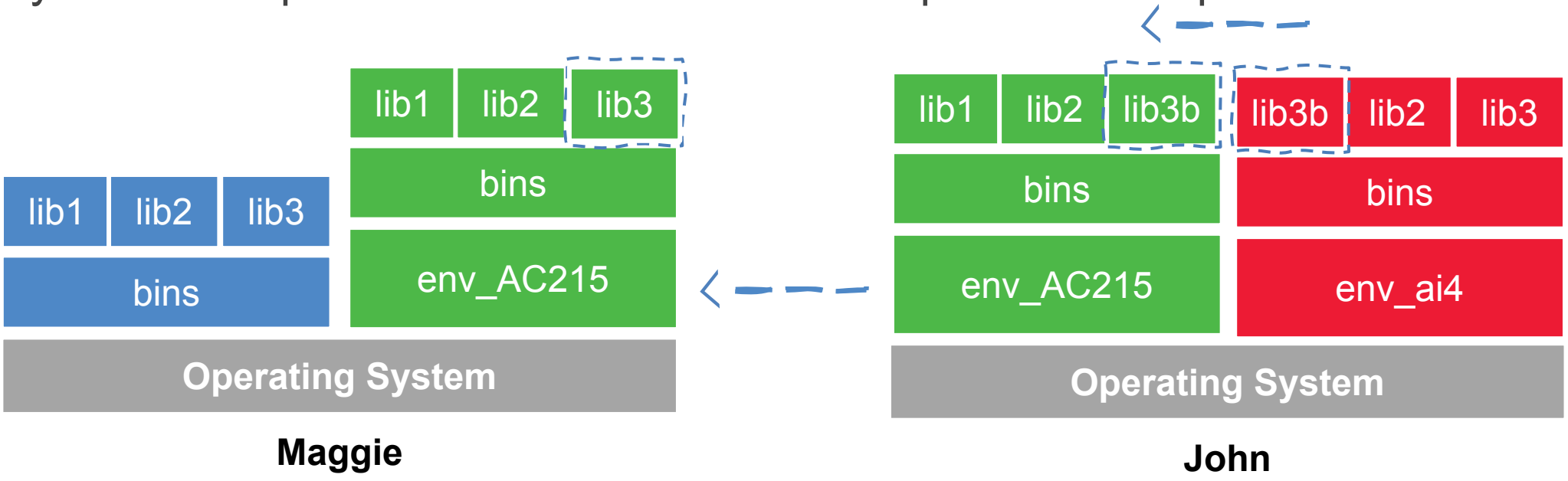
Why should we use virtual environment?

For the project, Maggie collaborates with John and shares their working environment by distributing a “configurations file” for the Conda environment.



Why should we use virtual environment?

John experiments with a new method they learned in another class and adds a new library to the working environment. After seeing tremendous improvements, they send Maggie back his code and a new “configuration file” (for conda env). They can now update their environment and replicate the experiment.



Why should we use virtual environment?

- Streamlines code development and usage.
- Isolates dependencies in separate "sandboxes" for easy switching between applications.
- Given an operating system and hardware, we can get the exact code environment set up using different technologies.

Virtual environments

Pros

- Reproducible Research: Enables consistent and replicable outcomes.
- Explicit Dependencies: Clearly defines all required software and packages.
- Enhanced Engineering Collaboration: Streamlines teamwork by standardizing environments.

Cons

- Setup Challenges: Initial environment configuration can be complex.
- Lack of Isolation: Does not completely isolate the working environment.
- OS Compatibility Issues: May not function consistently across different operating systems.

Creating Virtual Environments

- **venv (python3)**

  The default way to create virtual environments in python

- **conda**

  Is a package manager and environment manager for Data Scientists

- **pipenv**

  Production-ready tool that aims to bring the best of all packaging worlds to the Python world

- **mamba**

  Fast (C++) replacement for the Conda package manager that aims to offer quicker dependency resolution and installation

- **uv**

  An extremely fast Python package installer and resolver, written in Rust

We Will Be Using UV

Course Environment

For this class, we will be using **uv** as our primary virtual environment and package manager to standardise development.

Industry Standard

uv has rapidly become the modern industry benchmark, built in Rust for lightning-fast, highly reliable Python dependency and environment resolution.

venv

- Virtual environments manager embedded in Python
- Incorporated into broader tools such as pipenv
- Allow to install modules using pip package manager

venv

- create an environment within your project folder: `python3 -m venv your_env_name`
- it will add a folder called environment_name in your project directory
- activate environment: `source your_env_name/bin/activate`
- install requirements using: `pip install package_name==version`
- listing installed packages: `pip list`
- deactivate environment once done: `deactivate`
- removing a virtual environment: simply delete the `env_name` directory.
- to generate a list of all the Python packages that are currently installed in your environment: `pip freeze > requirements.txt`
- to install all the Python packages listed in a “configuration file”: `pip install -r requirements.txt`

`venv` does not have a built-in command to list all the virtual environments. You can use pew which needs to be installed

Conda

- Virtual environments manager embedded in Anaconda
- Allow to use both conda and pip to manage and install packages
- Base virtual environment comes pre-installed with various engineering and data science packages

Conda

How to use it:

- list all the environments:

```
conda env list
```

- create an environment:

```
conda create --name your_env_name python=3.7
```

it will add a folder located within your anaconda installation

```
/Users/your_username/[opt]/anaconda3/envs/your_env_name
```

- activate environment

```
conda activate your_env_name
```
(should appear in your shell)

- install requirements

```
conda install package_name=version
```

- deactivate environment once done

```
conda deactivate
```

- duplicate your environment using YAML file `conda env export > my_environment.yml`
- to recreate the environment now use `conda env create -f environment.yml`

A YAML (YAML Ain't Markup Language) file is similar to a dictionary. We will encounter many of them in this course

PipEnv

- Built on top of *venv*
- Fixes many shortcomings of *venv*
- Distinguish development vs. production environments
- Automatically keeps track of packages and package dependencies using a `Pipfile` & `Pipfile.lock`

All packages after [dev-packages] in Pipfile are for development

Pipfile is human readable and specify the packages and version constraints
Pipfile.lock not human readable and specifies the packages and their dependencies

PipEnv

How to use it:

- To install pipenv use: `pip install pipenv`
- Navigate to your project directory
- To create a new environment: `pipenv install`
- To activate the environment: `pipenv shell`
- To install a new package `pipenv install numpy`
- To sync from an existing Pipfile.lock: `pipenv sync`
- Exit the virtual environment: `exit`

uv

- An extremely fast Python package installer and resolver, written in Rust
- Drop-in replacement for pip and pip-tools
- Built-in virtual environment management
- Automatically keeps track of packages and package dependencies using *pyproject.toml* & *uv.lock*

Development dependencies are specified under [dependency-groups.dev] in pyproject.toml

pyproject.toml is human readable and specifies packages and version constraints

uv.lock is not human readable and specifies exact packages and their dependencies

How to Use **uv**

uv manages a Python project environment

It helps us answer:

- What packages does the project need?
- Which exact versions should be used?
- Where are packages installed?

Main idea:

uv replaces messy manual installs with a clean project workflow.

Basic Workflow

```
uv init --bare
uv add <package-name>
uv run <command>
```

- `uv init --bare` creates the project file
- `uv add` records and installs dependencies

Two ways to run code:

- `source .venv/bin/activate`
- `python cli.py`

or

- `uv run python cli.py`

runs code inside the project environment

The Two Key Files and a Key Directory

```
pyproject.toml
uv.lock
.venv/
```

- `pyproject.toml`: what the project needs
- `uv.lock`: exact versions selected
- `.venv/`: installed local environment

Example: Using uv for Simple Translate

```
uv init --bare
uv add googletrans==4.0.2 art
uv run python cli.py
```

This creates:

```
pyproject.toml    # project dependencies
uv.lock           # exact versions
.venv/            # installed environment
```

`uv add` records the libraries and installs them.

`uv run` runs the app inside the clean environment.

After the First Time

Once the project has already been set up, we should have:

`pyproject.toml`
`uv.lock`

Now we do not need to add the libraries again.

`uv sync` creates `.venv/`
installs exact libraries from `uv.lock`

`uv run python cli.py`

More on Virtual environments

Further readings

- UV: Official Documentation

  https://docs.astral.sh/uv/

- UV: An In-Depth Guide to Python's Fast and Ambitious New Package Manager

  https://www.saaspegasus.com/guides/uv-deep-dive/

- UV vs PIP: Revolutionizing Python Package Management

  https://medium.com/@sumakbn/uv-vs-pip-revolutionizing-python-package-management-576915e90f7e

- Python UV: The Ultimate Guide to the Fastest Python Package Manager

  https://www.datacamp.com/tutorial/python-uv

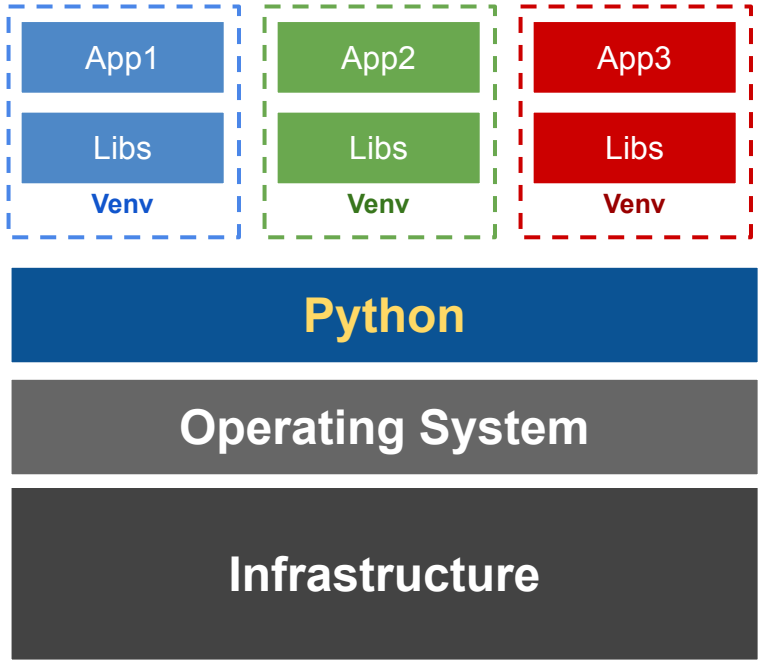
Virtual Environment Tutorial

- Let us run the simple-translate app using Virtual Environment
- For this we will do the following:
  - Create a VM Instance
  - SSH into the VM
  - Install dependencies: git, python
  - Download and run the simple-translate python script
- Full instructions can be found here.

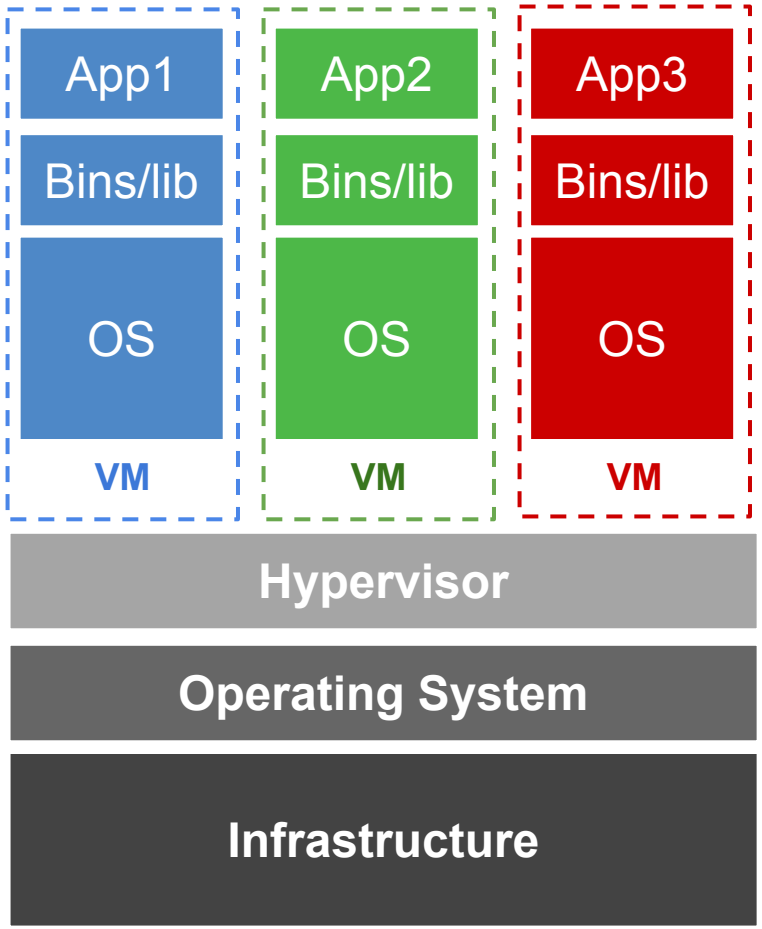
https://github.com/dlops-io/simple-translate?tab=readme-ov-file#installing-app-on-vm-using-uv-t2



Summary: Virtual Environments vs Virtual Machine



Virtual Environments



Virtualization

Logistics

- Survey
- Make project groups
- Setup & Installations



THANK YOU